



Product Requirement Document (PRD): DevPulse

1. Executive Summary

DevPulse is a personal developer workspace dashboard designed to aggregate tracking tools into a unified interface. It combines project management (projects and task boards) with a developer-focused logging system (daily coding metrics, standup notes, and repository data trackers).

2. System Architecture & Tech Stack

- **Frontend Framework:** React (Vite, JavaScript)
- **UI System:** Tailwind CSS + shadcn UI + Lucide React Icons
- **Backend Framework:** Python FastAPI (Asynchronous engine)
- **Database:** PostgreSQL (Relational database management system)
- **ORM (Object Relational Mapping):** SQLAlchemy or SQLModel (Async)
- **Authentication:** JWT (JSON Web Tokens) with OAuth2 password hashing

3. Core Features & Scope



Phase 1: Authentication & Security (The Foundation)

- User registration and secure login.
- Password hashing on the backend (using Passlib/Bcrypt).
- JWT token generation, token storage, and route protection in React.



Phase 2: Project & Task Management (The Relational Database Layer)

- **Projects:** Ability to create, update, and delete development projects.
- **Kanban Task Board:** A multi-column task board (To Do, In Progress, Done) assigned to specific projects.
- **Database Skill:** One-to-Many Relationships (One Project $\rightarrow$ Many Tasks).



Phase 3: Daily Metrics & Work Logs (The Time-Series Layer)

- **Daily Log:** A quick-submission journal where users log lines of code written, key bugs solved, hours worked, and a markdown text summary.
- **Activity Heatmap:** A visual matrix graph (similar to GitHub's contribution grid) fueled by user database logs.
- **Database Skill:** Composite relationships, tracking data points over a timeline.

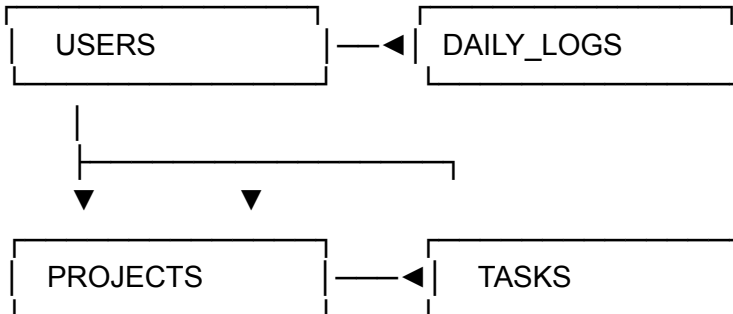


Phase 4: Mock Live Data Aggregator (The Third-Party API Layer)

- A section that periodically fetches external data (simulating background jobs fetching public repository stats like commits, stars, open issues) using FastAPI background tasks.

4. PostgreSQL Database Schema Concept (ERD Preview)

To design this cleanly, our PostgreSQL instance will manage 4 interconnected tables:



1. **users:** id (PK), email, hashed_password, created_at
2. **projects:** id (PK), user_id (FK), name, description, repository_url, created_at
3. **tasks:** id (PK), project_id (FK), title, status (todo/progress/done), priority, updated_at
4. **daily_logs:** id (PK), user_id (FK), date, lines_written, hours_spent, notes

5. UI/UX Design Goals (shadcn UI Layout)

- **Dashboard View:** Side navigation pane with views for "Overview", "Projects & Tasks", and "Daily Logs".
- **Theme:** Deep dark mode base theme utilizing custom monochrome slate variables (bg-zinc-950, text-zinc-50).
- **Components:** Leverage shadcn's <Card>, <Dialog> modals for forms, <Badge> tags, and <Table> layouts.